

Veri Besleme ve Temel SPL

Veri Besleme

Ubuntu lab makinesindeki (192.168.50.10) `/var/log/auth.log` dosyasını Splunk'a besledim. Bunu iki adımda yaptım:

- Add Data > Upload ile dosyanın o anki halini tek seferlik yükledim.
- Sonra Settings > Data Inputs > Files & Directories > Monitor ile aynı dosyayı sürekli izleyecek bir input kurdum, böylece dosyaya yeni satır her düştüğünde otomatik olarak Splunk'a akıyor.

- Index: `staj\_lab` (yeni oluşturdum)
- Sourcetype: `staj\_auth\_log` (manuel isim verdim)
- Source: `/var/log/auth.log`

`index=staj\_lab` sorgusuyla arayüzde eventlerin geldiğini gördüm. Zaman aralığını "All time" yaptım.

Ne anladım?:

Splunk'a veri sokmanın en basit yolu doğrudan dosya yüklemek (Upload); ama bu tek seferlik bir kopya alır, dosya değişse bile Splunk otomatik güncellemez. Sürekli izlenmesini istediğim bir log kaynağı için "Monitor" input kurmam gerekiyor. Bu, gerçek bir SIEM'in çalışma mantığına daha yakın.

SPL: Splunk'ın Sorgu Dili

1. Arama terimi — grep karşılığı

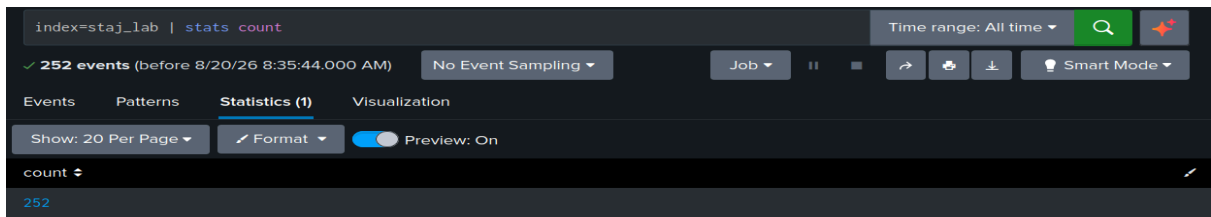
```
index=staj_lab "Failed password"
```

Belirli bir kelimeyi içeren event'leri filtreler. SPL'deki her sorgunun ilk adımı bu: önce ilgilendiğin olay kümesini daraltırsın, sonra üzerine stats, table gibi katmanlar eklersin.

2. stats count — wc -l karşılığı

```
index=staj_lab | stats count
```

Toplam event sayısını verir.



3. stats count by <alan> — sort | uniq -c karşılığı

index=staj_lab "Failed password" | stats count by src_ip

Bir alana göre gruplayıp sayar. Blue Team analizinin en temel kalıbı: "hangi kaynak, kaç kez" sorusunu tek satırda cevaplar (brute-force tespiti bunun üzerine kurulu).

4. top — stats count by + otomatik sıralama

index=staj_lab "Failed password" | top src_ip

Aynı işi yapar ama sonucu otomatik sıralar ve yüzdelik bilgi ekler.

5. table — seçili alanları düzenli göster

index=staj_lab | table _time, host, source

Ham log yerine sadece ilgilenilen sütunları tablo halinde gösterir; rapor/ekran görüntüsü için okunabilirliği artırır.

6. rex — manuel alan (field) çıkarma

index=staj_lab "Failed password" | rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | stats count by src_ip

src_ip alanı staj_auth_log sourcetype'ında otomatik çıkmadığı için gerekti.

Log satırı: Failed password for vboxuser from 192.168.50.20 port 36283 ssh2 — rex, from kelimesinden sonraki IP'yi yakalayıp yeni bir alan oluşturuyor. Gerçek vakalarda hazır alan çıkmayan log formatlarıyla sık karşılaşılır, bu yüzden önemli.

7. Zaman aralığı seçici — terminaldeki tarih/saat filtrelemenin karşılığı

Sorgu doğru olsa bile yanlış zaman aralığı seçilirse eksik sonuç alınabilir. Rapor yazmadan önce doğru aralıkta (örn. "All time") arama yapıldığından emin olunmalı.

Terminal	Splunk(SPL)
grep "Failed password"	arama terimi: "Failed password"
sort uniq -c	stats count by <alan>
wc -l	stats count
awk/cut ile alan ayıklama	rex ile alan (field) çıkarma
tarih/saat aralığı filtreleme	zaman aralığı seçici (Time Range Picker)

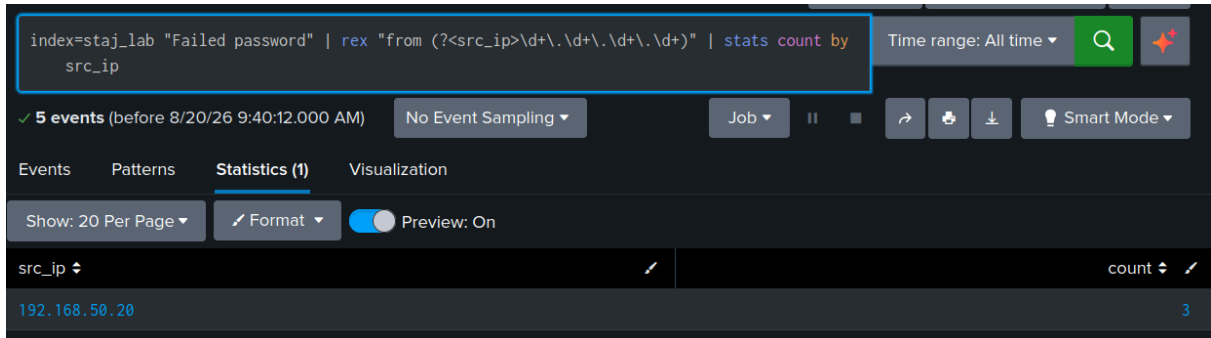
Pratik:

Geçen hafta terminalde "hangi IP kaç başarısız giriş yaptı" sorusunu grep/awk/sort/uniq -c zinciriyle cevaplamıştım. Bugün aynı soruyu SPL ile cevapladım:

1. Windows VM'den (192.168.50.20) Ubuntu'ya (192.168.50.10) bilerek 3 kez yanlış şifreyle SSH denemesi yaptım.
2. Ubuntu'da `sudo grep -a "Failed password" /var/log/auth.log` ile bu denemelerin log'a düştüğünü doğruladım.
3. Splunk'ta çalıştırdığım sorgu:

```
index=staj_lab "Failed password" | rex "from (?<src_ip>\d+\.\d+\.\d+\.\d+)" | stats count by src_ip
```

4. Sonuç:



Ne anladım?:

Aynı veriye, aynı soruya, iki farklı araçla ulaştım ve aynı cevabı aldım. SPL'in avantajı, terminaldeki 4-5 komutu tek satırda ve daha okunabilir şekilde ifade edebilmesi; ayrıca sonucu doğrudan görselleştirmeye (tablo, grafik) çevirebilmem.

Neden önemli?:

Bu, komut satırından SIEM'e geçişin somut kanıtı. İleride büyük veri hacimlerinde grep ile arama yapmak pratik olmayacak, Splunk bu ölçekte asıl işi görecek.